Advanced Day 2 — Scope & Closures

Student Document · Advanced JS · Session 2 of 15

★ ADVANCED JS — DAY 2 OF 15

60 min	30 min	15 min	15 min
TOTAL	TEACHING	HANDS-ON	Q&A

Today's Topics

#	Topic
1	Lexical (Static) Scope
2	The Scope Chain
3	What is a Closure?
4	Practical Closure Patterns (counter, private, memoize)
5	The Classic var-in-Loop Bug
6	IIFE — Immediately Invoked Function Expression
7	Why Closures Matter in Practice

PART 1 — FOLLOW ALONG WITH YOUR INSTRUCTOR

Topic 1 **Lexical (Static) Scope** Set by where code is written

A function's scope is determined by **where it is written** in the source code, not by where or how it is called. JS uses lexical scope only — never dynamic.

```
const city = "Jaipur";              // lives in the OUTER (global) scope

function outer() {
  const language = "Hindi";         // lives in outer's scope
  function inner() {
    const greeting = "Namaste";     // lives in inner's scope
```

```
    console.log(greeting, language, city);  // can see all three — looks outward
  }
  inner();
}

outer();   // "Namaste Hindi Jaipur"

// inner() can reach OUT to language and city because of WHERE it is written —
// nested inside outer(), which is nested inside the global scope.
```

Topic 2 **The Scope Chain** Walks one-way — outward

Step	Engine asks
1	Is the variable in the current scope?
2	If no → check outer scope
3	Repeat until global scope
4	Still nothing → `ReferenceError`

```
const a = "global a";

function outer() {
  const b = "outer b";
  function inner() {
    const c = "inner c";
    console.log(a);   // "global a"  ← walked up: inner → outer → global ✓
    console.log(b);   // "outer b"   ← walked up: inner → outer ✓
    console.log(c);   // "inner c"   ← found in current scope ✓
  }
  inner();
  // console.log(c);  // ReferenceError ← outer scope cannot see inner's variables
}
outer();
```

FOLLOW ALONG

```
const x = 1;
function a() {
  const y = 2;
  function b() {
    console.log(x + y);
  }
  b();
```

→ *prints?*

```
}
a();
```

Topic 3 What is a Closure?

Function + remembered scope

> **"A closure is a function that remembers the variables from the scope where it was created — even after that scope has finished."**

```
function makeGreeter(name) {        // outer function
  // 'name' lives in the outer EC

  return function () {              // inner function — referenced after outer returns
    console.log(`Namaste, ${name}!`);
  };
}

const greetPriya = makeGreeter("Priya");
const greetAarav = makeGreeter("Aarav");

greetPriya();    // "Namaste, Priya!"   ← still remembers name = "Priya"
greetAarav();    // "Namaste, Aarav!"   ← independent closure, name = "Aarav"

// makeGreeter has long since returned and its EC is gone from the call stack.
// But 'name' is still alive because the returned function "closed over" it.
// Each call to makeGreeter created a SEPARATE closure with its own 'name'.
```

⚠️ **Day 1 callback** — The outer EC is popped off the call stack the moment `makeGreeter` returns. But its variables aren't garbage-collected as long as something holds a reference to a function that closed over them. That's the closure.

Topic 4 Practical Closure Patterns

Counter · Private · Memoize

```
// 1. Counter — private state
function makeCounter() {
  let count = 0;                    // private — cannot be reached from outside
  return function () {
    count++;                        // each call mutates the SAME closed-over count
    return count;
  };
}

const c = makeCounter();
```

```
console.log(c());     // 1
console.log(c());     // 2
console.log(c());     // 3
// console.log(count) // ReferenceError — count is private to the closure

// 2. Private variables — bank account
function createAccount(initial) {
  let balance = initial;              // PRIVATE — no one outside can touch it
  return {
    deposit:    (amt) => balance += amt,
    withdraw:   (amt) => balance -= amt,
    getBalance: ()    => balance,
  };
}

const acc = createAccount(1000);
acc.deposit(500);
console.log(acc.getBalance());   // 1500
// acc.balance                   // undefined — truly private

// 3. Memoization — cache expensive results
function memoize(fn) {
  const cache = {};                   // closed-over cache, lives across calls
  return function (n) {
    if (n in cache) return cache[n];  // hit  → return cached
    cache[n] = fn(n);                 // miss → compute and store
    return cache[n];
  };
}

const slowSquare = (n) => { console.log("computing..."); return n * n; };
const fastSquare = memoize(slowSquare);
fastSquare(5);   // "computing..."  → 25
fastSquare(5);   // 25 (no log — served from cache)
```

Topic 5 **The var-in-Loop Bug** var shared · let fresh per iteration

```
// THE BUG — using var
for (var i = 0; i < 3; i++) {
  setTimeout(() => console.log(i), 100);
}
// Logs: 3, 3, 3   ← surprising!

// Why? var is FUNCTION-scoped. There is exactly ONE 'i' for the whole loop.
// All three callbacks closed over the SAME i.
// By the time setTimeout fires (100ms later), the loop has finished and i is 3.

// FIX 1 — let (block-scoped: each iteration gets a FRESH i)
for (let i = 0; i < 3; i++) {
```

```
  setTimeout(() => console.log(i), 100);
}
// Logs: 0, 1, 2

// FIX 2 — IIFE creating a new scope per iteration (the classic pre-ES6 fix)
for (var i = 0; i < 3; i++) {
  (function (j) {                    // j is a fresh parameter each iteration
    setTimeout(() => console.log(j), 100);
  })(i);
}
// Logs: 0, 1, 2

// FIX 3 — bind the value into a function call
for (var i = 0; i < 3; i++) {
  setTimeout(console.log.bind(null, i), 100);
}
// Logs: 0, 1, 2
```

FOLLOW ALONG

```
for (var k = 1; k <= 2; k++) {
  setTimeout(
    () => console.log(k),
    100
  );
}
```

→ *var → logs?*

Topic 6 **IIFE** Immediately Invoked Function Expression

```
// Basic IIFE — runs once, creates a private scope
(function () {
  const secret = "hidden";           // not visible outside
  console.log("IIFE ran");
})();

// IIFE with parameters
(function (city) {
  console.log(`Greetings from ${city}`);
})("Jaipur");

// Arrow IIFE (modern)
(() => {
  const x = 42;
  console.log(x);
})();
```

```
// Pre-ES6 module pattern — the most common historical use
const counterModule = (function () {
  let count = 0;                    // private (closure)
  return {
    inc: () => ++count,
    get: () => count,
  };
})();

counterModule.inc();
counterModule.inc();
console.log(counterModule.get());    // 2
// counterModule.count                // undefined — private
```

Topic 7 **Why Closures Matter** Real-world appearances

Where you'll meet closures	What they do
React `useState`, `useEffect`	Hold component state across renders
Event handlers using outer values	Closure captures the surrounding scope
Debounce / throttle (Day 15)	Store last-call time in a closure
Module pattern (pre-ES6)	Private vars via IIFE closure
Currying / partial app (Day 13)	Each return is a new closure

PART 2 — HANDS-ON EXERCISE (15 MIN)

Task 1 Build a Counter

- Write a function `makeCounter()` that returns a function.
- The returned function, each time it is called, returns the next integer starting from 1.
- Create TWO independent counters with `makeCounter()` and verify they don't interfere.
- In a comment, explain WHERE the count variable lives between calls.

Task 2 Fix the var-in-Loop Bug

- Type this snippet exactly: `for (var i = 1; i <= 3; i++) { setTimeout(() => console.log(i), 100); }`
- Predict the output. Run it. Note the actual output.
- Fix it by changing ONE keyword. Run again, verify it logs 1, 2, 3.

- In a comment, explain why `var` produced the surprising output and `let` fixes it.

Task 3 **Private Bank Balance**

- Write a function `createAccount(initial)` that returns an object with three methods: `deposit(amount)`, `withdraw(amount)`, `getBalance()`.
- `balance` must be PRIVATE — not accessible as `account.balance`.
- Test with: open an account with 1000, deposit 500, withdraw 200, log the balance.
- Verify `account.balance` is `undefined`.

Bonus **Build a Memoizer**

- Write a function `memoize(fn)` that returns a new function caching results of `fn` by argument value.
- Use it to wrap an `expensiveSquare(n)` that logs `"computing..."` and returns `n * n`.
- Call the memoized version with `5` twice, then `10` once, then `5` again. Note when the log fires.
- In a comment, explain where the cache lives.

PART 3 — VISUAL SUMMARY

Scope — Quick

Type	Set by	Searches
Lexical	Where written	Outward only
Block	`{ }`	`let` / `const` only
Function	`function` body	`var` and `let` / `const`

Closure — Mental Model

Stage	What happens
Outer function called	Outer EC pushed
Inner function returned	Inner holds reference to outer's vars
Outer EC popped	Vars stay alive — closure
Outside code calls inner	Inner reads/writes outer's surviving vars

var-in-Loop — Cheat

Keyword	One i or many?	Logs
`var i`	One shared i	Final value × N
`let i`	Fresh i per loop	0, 1, 2, ...

Common Closure Uses

Pattern	Hidden in closure
Counter	`count`
Bank account	`balance`
Memoizer	`cache`
Debounce	`timerId`

HOMEWORK

Practice tasks before next class · Advanced Day 2

1. Write `multiplier(factor)` that returns a function multiplying its argument by `factor`. Create `double = multiplier(2)` and `triple = multiplier(3)`. Verify both work independently.
2. Re-do the var-in-loop bug with a `for...of` over `[10, 20, 30]` and `setTimeout`. Use `let`. Predict, then verify.
3. Take the bank-account closure and add a `transactionCount` private variable that increments on every deposit/withdraw. Add a `getTransactionCount()` method.
4. Write `once(fn)` — a closure that takes a function and returns a wrapped version that only runs the FIRST time it's called. Subsequent calls return the cached first result.
5. Read: `javascript.info/closure` (full chapter) and `javascript.info/var` if you skipped it yesterday.
6. *(Bonus)* Watch Akshay Saini's "Namaste JS" Episode 10 — Closures. ~15 minutes.